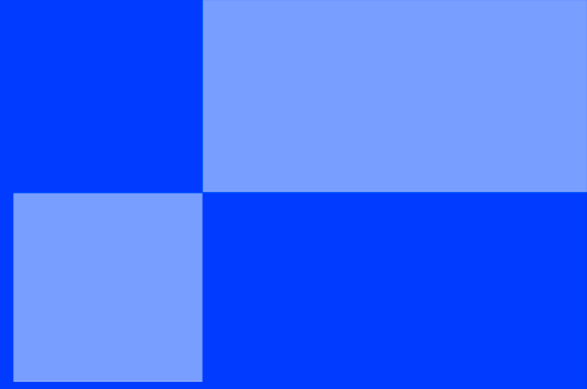




CRASH & LEARN

< MANDATORY SUBTITLE HERE />

CRASH & LEARN

Crash&Learn Grand Prix — Usage Instructions

Quick Start

```
# Build the Docker services  
docker compose build  
  
# Smoke test (runs demo.py)  
docker compose run --rm sim
```

Tests & Demo

```
# Run the built-in demo (uses submission/agent.py if present, fallback controller otherwise)  
docker compose run --rm sim
```

```
# Run contract unit tests  
docker compose run --rm sim python test_contract.py
```

```
# Run integration physics tests  
docker compose run --rm sim python test_integration.py
```

Recording & Visualization (Record + Replay)

The simulation uses a **record-then-replay** workflow: `sim_recorder.py` simulates the race at maximum speed and exports an `.npz` file, while `viz_replay.py` renders the race with interactive controls.

1. Record an episode

```
# Solo run (default random driver, 2 laps)
docker compose run --rm recorder

# Custom run (name, cars, laps)
docker compose run --rm recorder python sim_recorder.py --steps 1000 --num-cars 2 --out
/app/my_race.npz

# Multi-car recording (up to 4 cars)
docker compose run --rm recorder python sim_recorder.py --steps 1000 --num-cars 4

# Multi-car on a specific map
docker compose run --rm recorder python sim_recorder.py --steps 2000 --num-cars 4 --laps 2 --map
Spa --out /app/spa_grand_crash.npz

# With your trained submission
docker compose run --rm recorder python sim_recorder.py --steps 1000 --submission submission/
```

2. Replay recorded episodes (requires display)

```
# Interactive replay of the latest episode (requires X11)
xhost +local:docker
docker compose run --rm -e DISPLAY=$DISPLAY -v /tmp/.X11-unix:/tmp/.X11-unix replay

# Replay a specific episode with camera following car 0
docker compose run --rm -e DISPLAY=$DISPLAY -v /tmp/.X11-unix:/tmp/.X11-unix replay python
viz_replay.py --npz /app/my_race.npz --follow
```

Replay controls

Key / Action	Description
Space / Play / Pause	Play / pause in real time
← / →	Step backward / forward 1 frame
Shift+← / Shift+→	Step backward / forward 10 frames
Slider	Jump to any step
Mouse Wheel	Zoom in / out centered on mouse cursor

Key / Action	Description
Click + Drag	Pan across the circuit
Double-Click / R	Reset camera and zoom

Ranking. Each car starts from its own grid slot and finishes on that same slot, so every car covers exactly the same distance. The ranking compares distance covered, not on-screen position: a back-row car can lead the ranking while looking behind.

Export MP4 video Record the full replay as an MP4 video (headless, no display needed):

```
# Export the latest episode as MP4
docker compose run --rm replay python viz_replay.py --record /app/video.mp4 --no-telemetry

# Export with a specific episode
docker compose run --rm replay python viz_replay.py --npz /app/my_race.npz --record /app/video.mp4 --no-telemetry

# Or locally (headless, no display)
python3 resources/4students/simulation/viz_replay.py --npz resources/4students/simulation/recordings/episode_0001.npz --record output.mp4 --no-telemetry
```

The video plays at simulation speed (20fps = `sim_dt`).

Training Your Agent

You create your own training pipeline (`train.py`, `env.py`, etc.).

The image ships what the simulation and the submission need, and nothing else:

`numpy`, `numba`, `scipy`, `Pillow`, `pyyaml`, `imageio`, `matplotlib`, `tensorboard`, `pytest`,
plus the pinned ONNX toolchain `torch` / `onnx` / `onnxscript` / `onnxruntime`.

Your training stack is yours to install — the framework is your call. Add it to the image (or to your own layer on top of it):

```
pip install <your-training-stack>
```

Any framework works as long as what you submit is a `model.onnx` at **opset ≤ 17** that the pinned `onnxruntime` can load, and `agent.py` imports nothing but `onnxruntime` and `numpy`. `torch` is already in the image and exports ONNX natively; anything else (TensorFlow/Keras, JAX, scikit-learn, ...) needs its own ONNX exporter, which you install alongside it.

Do not change the pinned `torch` / `onnx` / `onnxruntime` versions: they are what makes your exported model loadable by the tournament runtime.

`torch` is installed as a **CPU-only** build: the simulation needs no GPU, and the CUDA runtime would add ~2.7 GB to the image for nothing. If you want to train on a GPU, that is an opt-in on your side — reinstall `torch` from the CUDA index and expose your device to the container. A CPU-trained submission is perfectly acceptable.

```
# Run your training script
docker compose run --rm sim python train.py

# Launch TensorBoard in background (open http://localhost:6006)
docker compose up -d tensorboard
```

Observing opponents

`obs["opponents"]` always holds the four fixed slots `0..3` (your own slot and any slot without a car are zero-filled). Each slot is a dict, expressed **in your car's frame**:

```
obs["opponents"][slot] = {
    "x_rel": float, # m, ego frame (forward = +x)
    "y_rel": float, # m, ego frame (left = +y)
    "yaw_rel": float, # rad, relative heading, wrapped to [-pi, pi]
    "speed": float, # m/s, that opponent's scalar speed
```

```
"progress": float, # [0.0, 1.0), that opponent's own-start lap phase
"lap_count": int, # that opponent's completed laps
"status": int, # 0 = inactive / DNF, 1 = active, 2 = finished
}
```

Only a slot with `status == 1` carries meaningful geometry. Any other slot — empty, your own, DNF (0) or already finished (2) — is zero-filled, which would read as a car sitting exactly at your position. Filter on `status == 1` before using `x_rel` / `y_rel`.

Submission structure

Local testing, a single agent To run `demo.py` or `sim_recorder.py` against your own agent, a flat `submission/` folder is enough: the loader treats it as a single team and replicates it across every car in self-play.

```
submission/
├── model.onnx
└── agent.py
```

Project delivery The delivery format is a different one: one subfolder per driver, plus a `champion` folder from Part 3 onwards. It is described in [the project subject](#).

`agent.py` must declare an `Agent` class exposing `predict(obs: dict, info: dict) -> tuple[float, float]` returning `(target_speed, steering)`. (See [the project subject](#) for the full specification and the ONNX export constraints).

Vérifier une submission

`verify_submission.py` valide le format et le contenu d'une submission avant le tournoi. Deux modes :

Mode single agent (par défaut) : vérifie les imports, l'opset ONNX ≤ 17 , le contrat `predict()`, le chargement via `agent_loader`. Les fichiers `agent.py`, `model.onnx` et si présent le `model.onnx.data` doivent être dans votre dossier `submission/`.

```
docker compose run --rm sim python verify_submission.py --submission submission/
```

Mode `--grandprix` : vérifie la structure de livraison (dossiers `driver1/`, `driver2/`, etc., chacun avec `agent.py` + `model.onnx`, noms uniques). Les fichiers de chaque voiture doivent être dans des sous-dossiers différents dans `submission/`.

```
docker compose run --rm sim python verify_submission.py --submission submission/ --grandprix
```

Track width

maps/<Name>/<Name>_centerline.csv holds `x_m`, `y_m`, `w_tr_right_m`, `w_tr_left_m`: the last two columns are the **half-widths** of the corridor, right and left of the centerline, measured per waypoint on the occupancy image by `maps/measure_widths.py`.

They are **not** the same on every circuit. The corridor ranges from about **1.44 m (Montreal)** to **2.60 m (Brands Hatch)**, and varies along a lap. Upstream (`f1tenth_racetracks`) shipped a flat `1.1, 1.1` on every row, inherited from a 1:10 downscale done at a fixed 2.20 m; the per-circuit rescaling that followed never corrected those columns. They are now measured values, so an agent can rely on them.

A car is declared **DNF for leaving the track** once its centre passes the local half-width plus a 0.50 m tolerance (`_OFFTRACK_MARGIN_M`).

24 circuits are available under `maps/<Name>/` : Austin, BrandsHatch, Budapest, Catalunya, Hockenheim, IMS, Melbourne, MexicoCity, Montreal, Monza, MoscowRaceway, Nuerburgring, Oschersleben, Sakhir, SaoPaulo, Sepang, Shanghai, Silverstone, Sochi, Spa, Spielberg, YasMarina, Zandvoort, example.

```
from env_simulation import set_map, get_available_maps, reset, get_obs, apply_action,
simulation_step, get_step_info, close

# List all available tracks
print(get_available_maps())

# Switch active track
set_map("Spa")

# Run simulation on the selected map
reset(num_cars=1)
for _ in range(500):
    obs = get_obs(0)
    apply_action(0, 3.0, 0.0)
    simulation_step()
    info = get_step_info()
    if info["lap_complete"][0]:
        break
close()
```

Counting laps. `info["lap_complete"][id]` is an edge signal: it is `True` only on the step a lap is booked (including the last one). For the number of laps already done, read the absolute counter `obs["lap_count"]` — it lives in `get_obs()`, not in `get_step_info()`.

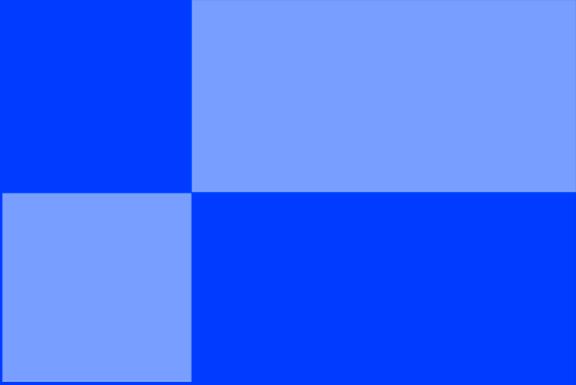
Arguments Reference

`sim_recorder.py`

Argument	Default	Description
<code>--steps N</code>	2000	Maximum decision steps
<code>--laps N</code>	2	Target completed laps per car
<code>--num-cars N</code>	1	Number of cars on track (1–4)
<code>--map NAME</code>	example	Map/track name (Austin, BrandsHatch, Budapest, Catalunya, Hockenheim, IMS, Melbourne, MexicoCity, Montreal, Monza, MoscowRaceway, Nuerburgring, Oschersleben, Sakhir, SaoPaulo, Sepang, Shanghai, Silverstone, Sochi, Spa, Spielberg, YasMarina, Zandvoort)
<code>--submission PATH</code>	""	Path to a submission directory or parent folder of team subdirectories
<code>--model PATH</code>	""	Direct path to an ONNX model file for car 0
<code>--controller {auto,submission,random,onnx,pure_pursuit}</code>	auto	Policy controller type
<code>--out PATH</code>	auto	Output .npz file path (auto: recordings/episode_XXXX.npz)

`viz_replay.py`

Argument	Default	Description
<code>--npz PATH</code>	""	Path to .npz recording file
<code>--follow</code>	false	Follow car 0 with camera
<code>--record PATH</code>	""	Export replay as MP4 video (headless)
<code>--telemetry</code>	true	Show telemetry overlay (default: on)
<code>--no-telemetry</code>	false	Hide telemetry overlay for cleaner video
<code>--no-display</code>	false	Headless mode (saves snapshot PNG)



{EPITECH}